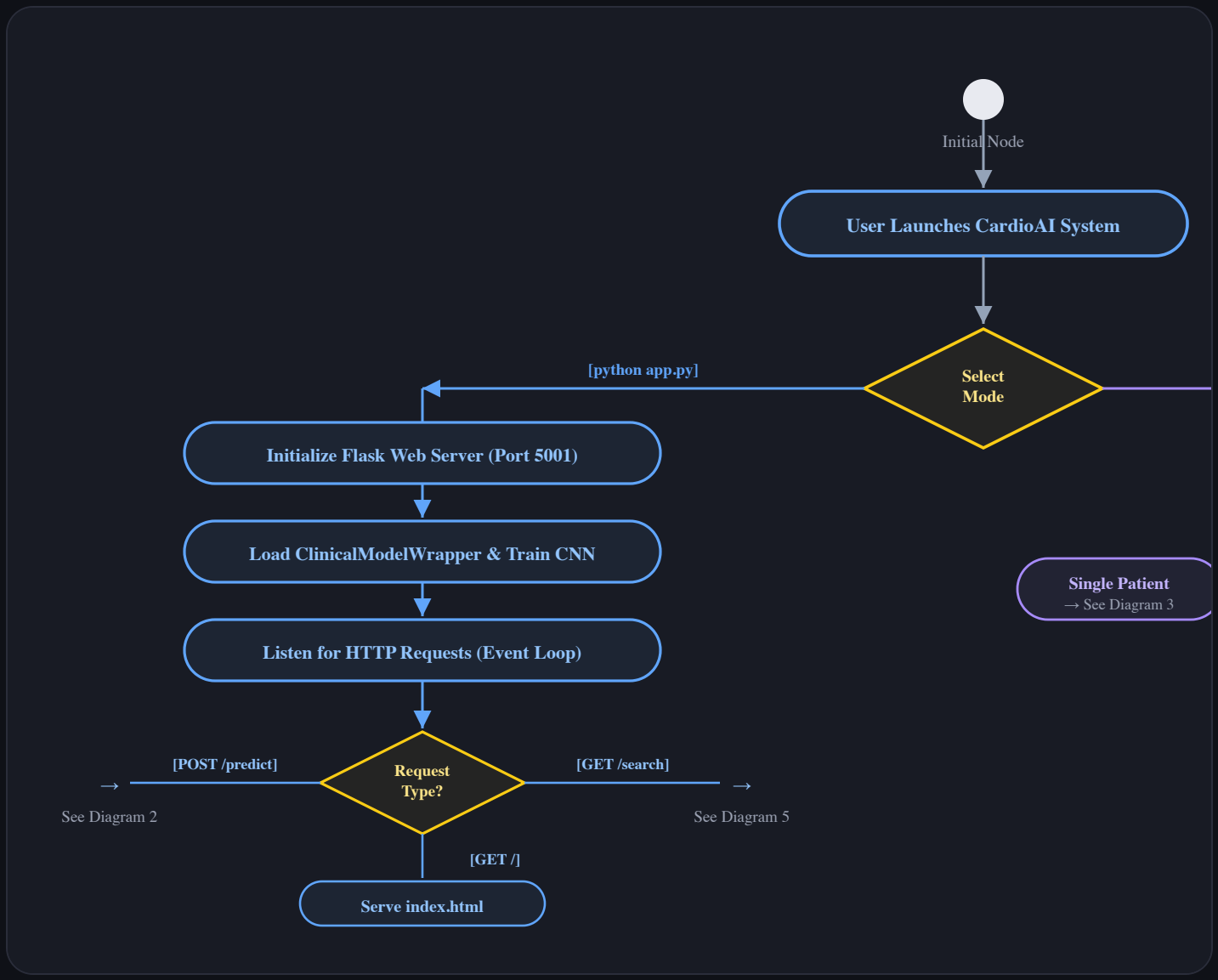
UML ACTIVITY DIAGRAM – SOFTWARE ENGINEERING

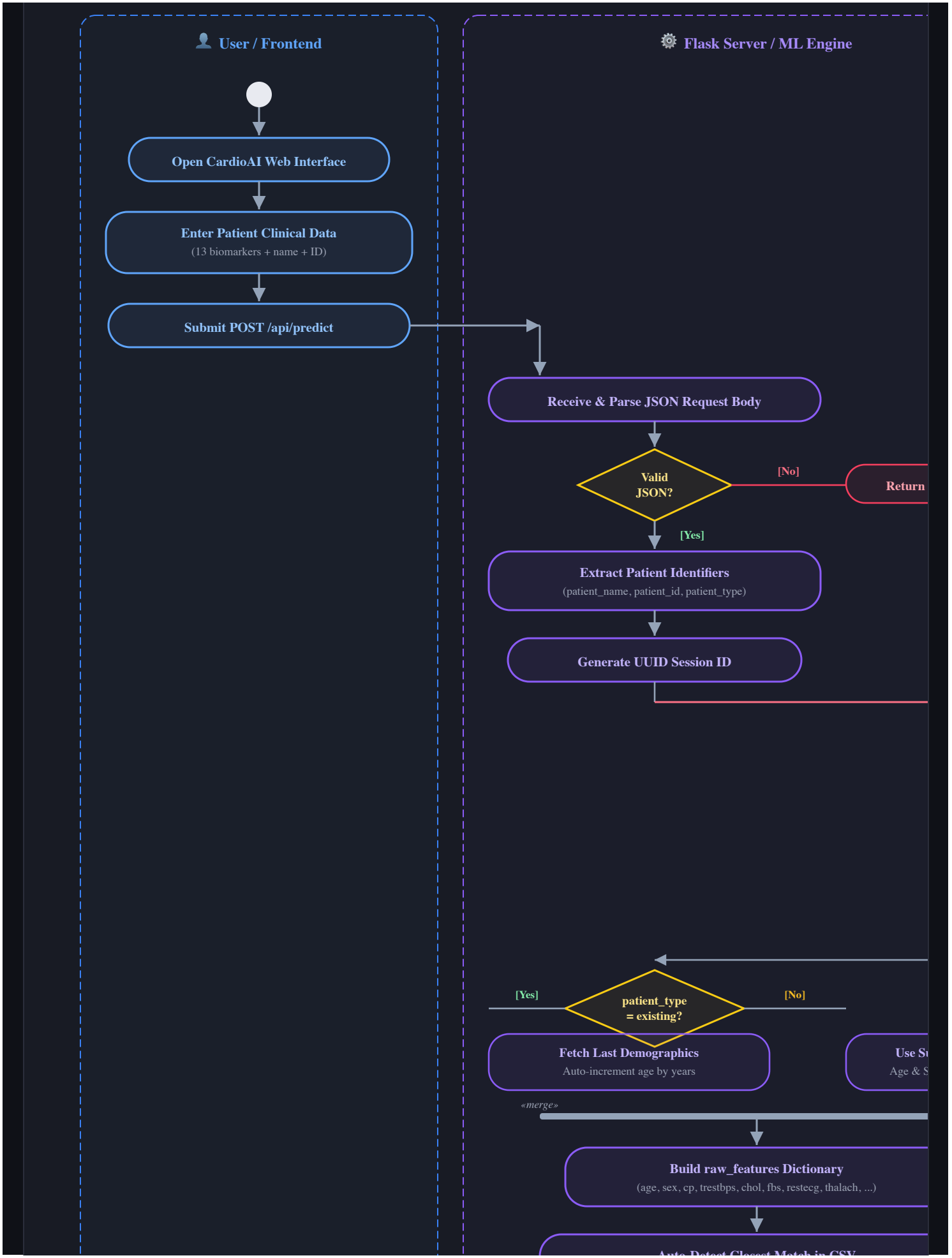
CardioAI – Activity Diagram

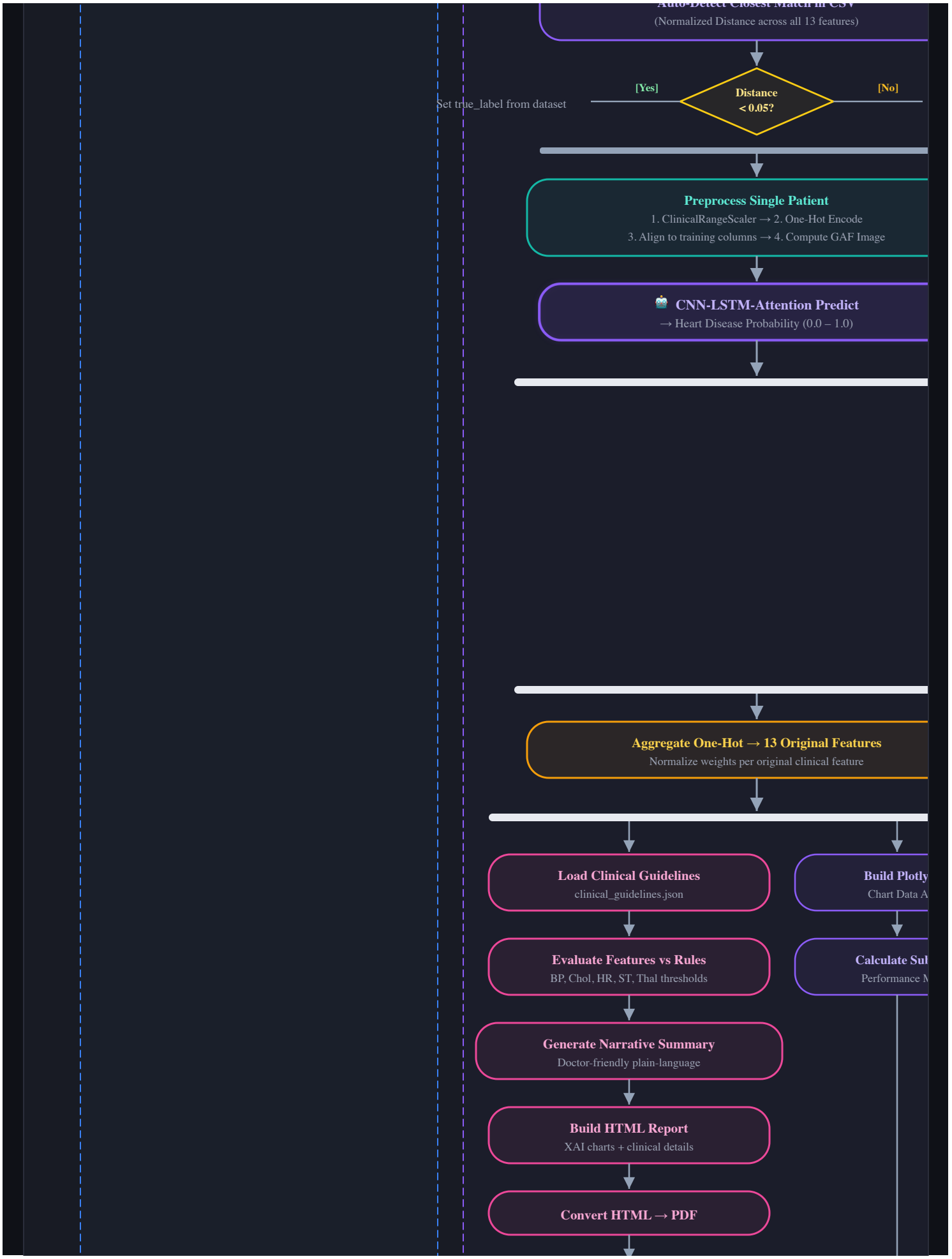
Explainable Heart Disease Prediction System · Complete System Workflow (Start to End)

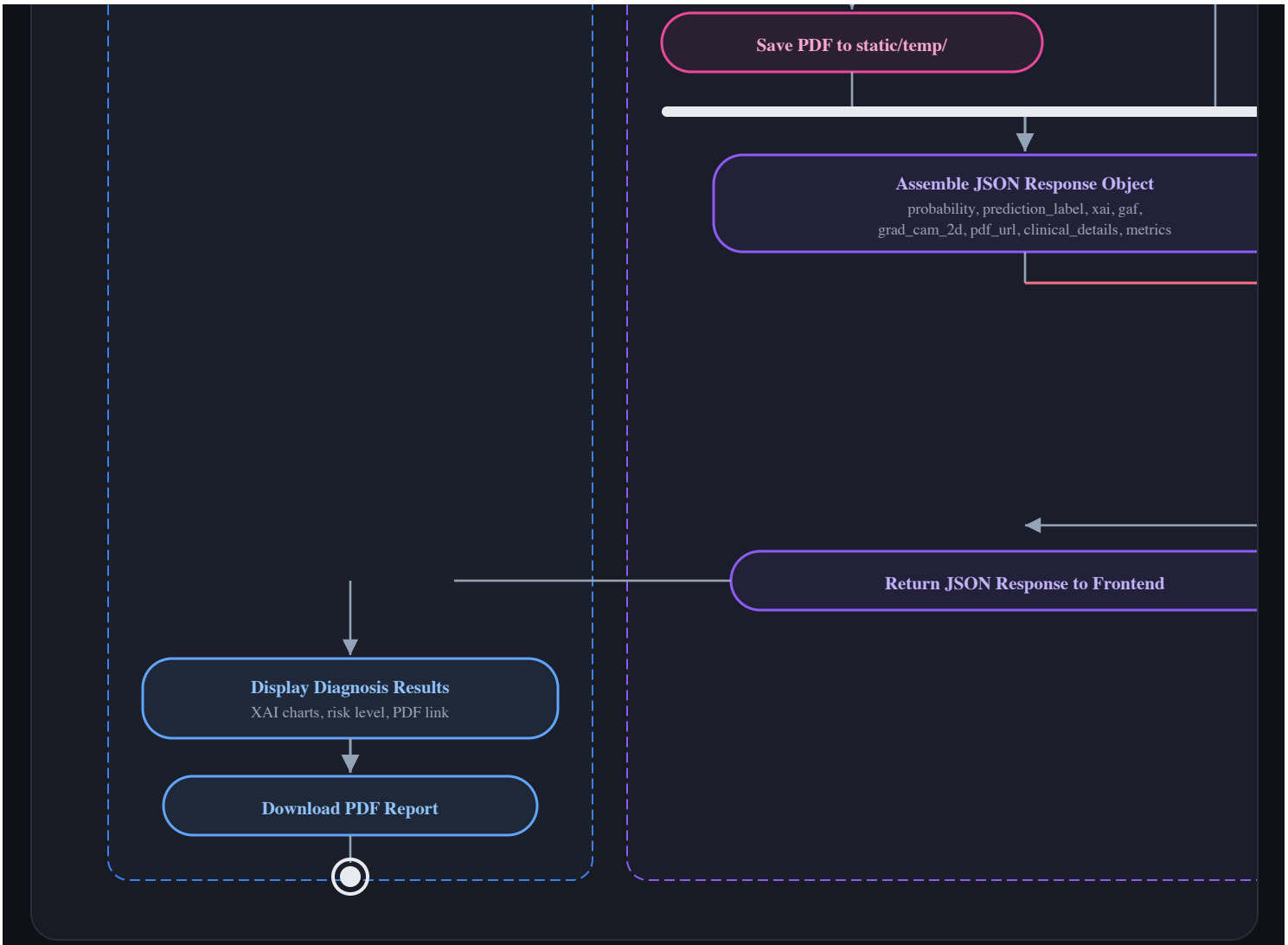
1 System-Level Activity Diagram — Top-Level Overview



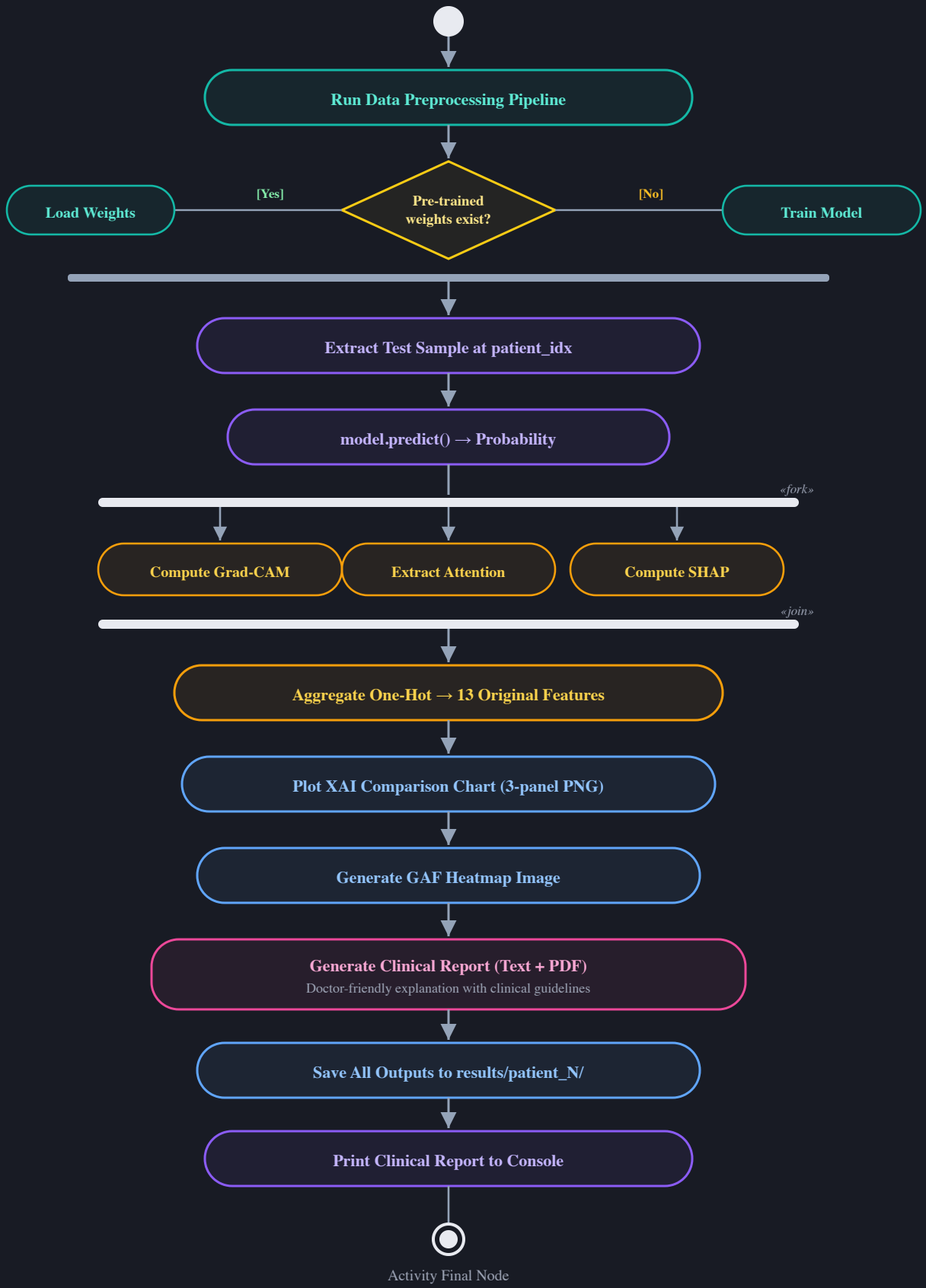
2 Web Application — Heart Disease Prediction Activity (POST /api/predict)


User / Frontend
Flask Server / ML Engine
Open CardioAI Web Interface
Enter Patient Clinical Data
(13 biomarkers + name + ID)
Submit POST /api/predict
Receive & Parse JSON Request Body
Valid JSON?
[No]
Return
[Yes]
Extract Patient Identifiers
(patient_name, patient_id, patient_type)
Generate UUID Session ID
[Yes]
patient_type = existing?
[No]
Fetch Last Demographics
Auto-increment age by years
«merge»
Build raw_features Dictionary
(age, sex, cp, trestbps, chol, fbs, restecg, thalach, ...)


(Normalized Distance across all 13 features)
[Yes]
Distance
< 0.05?
[No]
Set true_label from dataset
Preprocess Single Patient
1. ClinicalRangeScaler → 2. One-Hot Encode
3. Align to training columns → 4. Compute GAF Image
CNN-LSTM-Attention Predict
→ Heart Disease Probability (0.0 – 1.0)
Aggregate One-Hot → 13 Original Features
Normalize weights per original clinical feature
Load Clinical Guidelines
clinical_guidelines.json
Evaluate Features vs Rules
BP, Chol, HR, ST, Thal thresholds
Generate Narrative Summary
Doctor-friendly plain-language
Build HTML Report
XAI charts + clinical details
Convert HTML → PDF

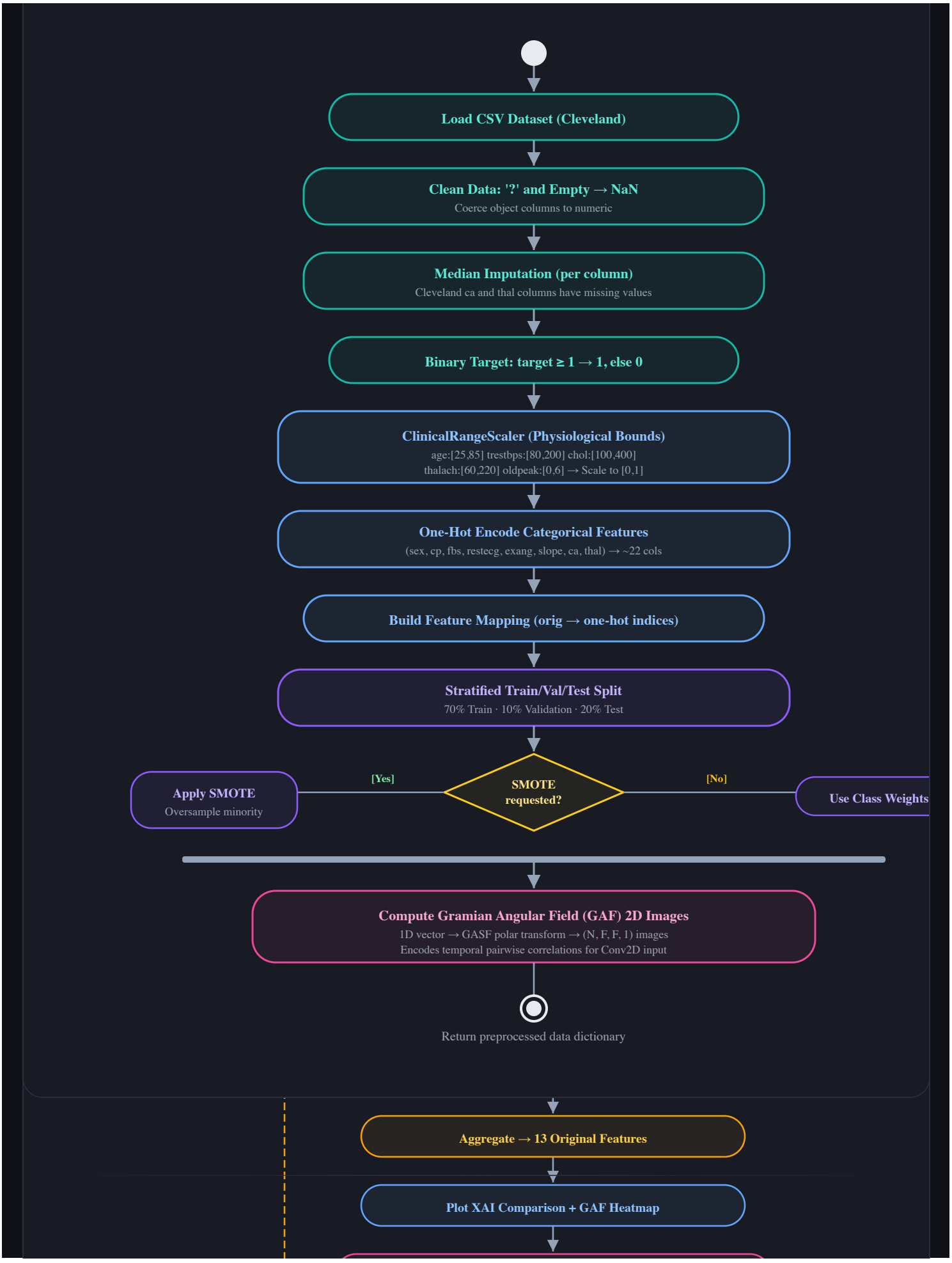

Save PDF to static/temp/
Assemble JSON Response Object
probability, prediction_label, xai, gaf,
grad_cam_2d, pdf_url, clinical_details, metrics
Return JSON Response to Frontend
Display Diagnosis Results
XAI charts, risk level, PDF link
Download PDF Report

3 CLI Pipeline — Single Patient Explanation Mode (--patient_idx)


Run Data Preprocessing Pipeline
Pre-trained weights exist?
[Yes]
Load Weights
[No]
Train Model
Extract Test Sample at patient_idx
model.predict() → Probability
«fork»
Compute Grad-CAM
Extract Attention
Compute SHAP
«join»
Aggregate One-Hot → 13 Original Features
Plot XAI Comparison Chart (3-panel PNG)
Generate GAF Heatmap Image
Generate Clinical Report (Text + PDF)
Doctor-friendly explanation with clinical guidelines
Save All Outputs to results/patient_N/
Print Clinical Report to Console
Activity Final Node

4 CLI Pipeline — Full Training & Evaluation Mode (python main.py)

5 Data Preprocessing Sub-Activity Diagram (preprocess_pipeline)


Load CSV Dataset (Cleveland)
Clean Data: '?' and Empty → NaN
Coerce object columns to numeric
Median Imputation (per column)
Cleveland ca and thal columns have missing values
Binary Target: target ≥ 1 → 1, else 0
ClinicalRangeScaler (Physiological Bounds)
age:[25,85] trestbps:[80,200] chol:[100,400]
thalach:[60,220] oldpeak:[0,6] → Scale to [0,1]
One-Hot Encode Categorical Features
(sex, cp, fbs, restecg, exang, slope, ca, thal) → ~22 cols
Build Feature Mapping (orig → one-hot indices)
Stratified Train/Val/Test Split
70% Train · 10% Validation · 20% Test
SMOTE requested?
[Yes]
Apply SMOTE
Oversample minority
[No]
Use Class Weights
Compute Gramian Angular Field (GAF) 2D Images
1D vector → GASF polar transform → (N, F, F, 1) images
Encodes temporal pairwise correlations for Conv2D input
Return preprocessed data dictionary
Aggregate → 13 Original Features
Plot XAI Comparison + GAF Heatmap

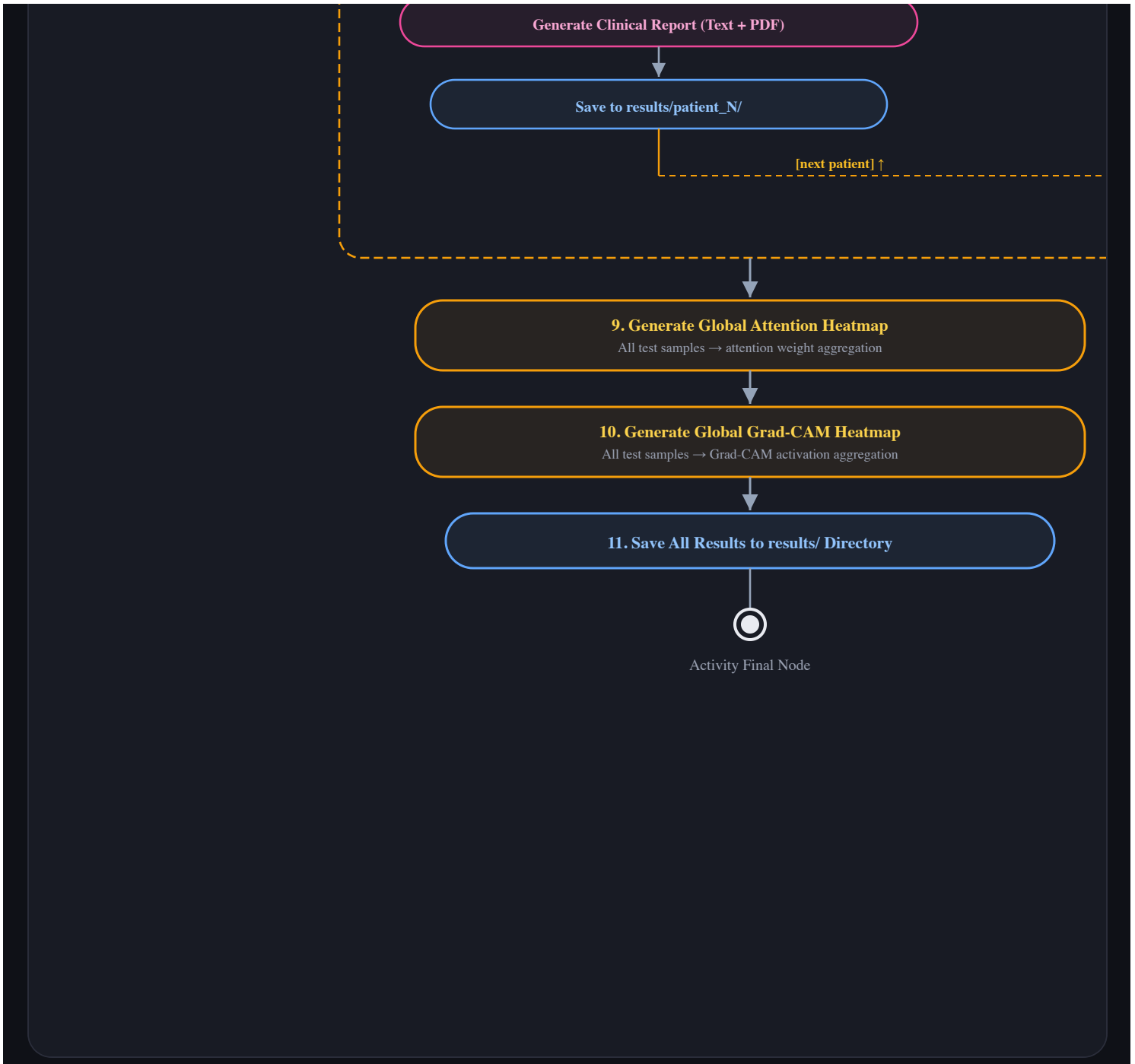

Generate Clinical Report (Text + PDF)
Save to results/patient_N/
[next patient] ↑
9. Generate Global Attention Heatmap
All test samples → attention weight aggregation
10. Generate Global Grad-CAM Heatmap
All test samples → Grad-CAM activation aggregation
11. Save All Results to results/ Directory
Activity Final Node



Diagram 1: System-Level Overview

Diagram 2: Web Prediction Flow

- The system starts with a **decision node** selecting between two execution modes: **Web Application** (`python app.py`) or **CLI Training Pipeline** (`python main.py`)
- The web app mode initializes a Flask server on port 5001 and loads the `ClinicalModelWrapper` which trains or loads the CNN-LSTM-Attention model
- The CLI mode parses arguments via `argparse` with a further decision for `--patient_idx` (single patient) vs full pipeline

- **Swimlanes:** 4 partitions — User/Frontend, Flask/ML, XAI Module, MySQL Database
- **Patient Deduplication:** `get_or_create_patient()` checks if hospital_patient_id exists; creates new record if not
- **Existing Patients:** Auto-increments age based on time since last visit using `get_patient_last_demographics()`
- **Ground Truth Matching:** Computes normalized distance to Cleveland CSV dataset; if distance < 0.05, sets true_label from dataset
- **Fork/Join:** Three XAI methods (Grad-CAM, Attention, SHAP) execute in parallel, then join before aggregation
- **Second Fork:** PDF report generation and API response assembly happen in parallel, then join for DB persistence
- **Database Writes:** Two INSERTs — `clinical_records` (14 params) and `ai_diagnoses` (probability, class, top 3 drivers, PDF path)

📋 Diagram 3: Single Patient Explanation

- Triggered via `python main.py --patient_idx N` where N is a test set index
- **Decision:** Checks for pre-trained weights file; if found, loads them; otherwise trains the model first
- **Fork/Join:** Grad-CAM, Attention, and SHAP computed in parallel
- **Outputs:** XAI comparison chart (PNG), GAF heatmap (PNG), clinical report (TXT), PDF report — all saved to `results/patient_N/`

📋 Diagram 4: Full Training Pipeline

- **11 sequential phases** covering EDA → Preprocessing → GAF Demo → Tuning → Training → Ablation → CV → XAI Profiling → Global Heatmaps
- **Fork (3-way):** CNN, CNN-LSTM, CNN-LSTM-Attention models trained in parallel
- **Two optional decision nodes:** `--tune` enables Keras Tuner RandomSearch; `--cv` enables 5-fold cross-validation
- **Loop region:** XAI profiling iterates over selected True Positive and True Negative test patients
- **Ablation Study:** Compares all 3 models across Accuracy, Precision, Recall, F1-Score, ROC-AUC; saves CSV + barplot

📋 Diagram 5: Data Preprocessing

- Implements `preprocess_pipeline()` from `src/preprocessing.py`
- **ClinicalRangeScaler:** Uses fixed physiological boundaries instead of dataset-dependent statistics for out-of-sample consistency
- **One-Hot Encoding:** Expands 8 categorical features into ~22 binary columns
- **Feature Mapping:** Stores original-to-processed column index map, critical for XAI aggregation back to 13 clinical features
- **SMOTE Decision:** Conditionally applies Synthetic Minority Oversampling; falls back to class weights if imblearn is unavailable
- **GAF Transform:** Converts 1D feature vectors to 2D Gramian Angular Summation Field images for CNN

📋 UML Notation Guide

- **Initial Node (●):** Filled black circle — entry point of the activity
- **Activity Final Node (◉):** Filled circle inside a circle — termination point
- **Action State:** Rounded rectangle — represents a single activity/operation
- **Decision Node (◇):** Diamond — branching point with guard conditions
- **Fork Bar (▬ white):** Thick white bar — splits flow into concurrent threads
- **Join Bar (▬ white):** Thick white bar — synchronizes concurrent threads
- **Merge Bar (▬ gray):** Gray bar — merges alternate paths back together

Conv2D input

- **Guard Conditions [text]:** Boolean expressions on transitions (e.g., [Yes], [No])
- **Swimlanes:** Vertical partitions assigning actions to responsible actors/components
- **Loop Region:** Dashed boundary — iterative repetition of enclosed activities